```
import itertools

def check_entailment_and_biconditional(expression_A, expression_B):
    """
    Evaluates three boolean variables A, B, C (or generic props)
    Checks if A entails B (A -> B is a tautology / always True when A is True)
    Prints a truth table and checks bidirectional equivalence (A <-> B)
    """
    variables = ['A', 'B', 'C']
    print(f"| { ' | '.join(variables) } | A (KB) | B (Query) | A => B | B => A | A <=> B | Entailed? |")
    print("|-" + "-|-".join(['-'*len(v) for v in variables]) + "-|--------|-----------|--------|--------|---------|-----------|")

    all_entails = True
    all_biconditional = True

    for values in itertools.product([True, False], repeat=3):
        env = dict(zip(variables, values))

        # Unpack values for readability inside expressions
        A, B, C = env['A'], env['B'], env['C']

        # Define your logic for A (Knowledge Base) and B (Query) here:
        # Example: A entails B where A = (A and B), B = B (or customize)
        val_A = eval(expression_A, {}, env)
        val_B = eval(expression_B, {}, env)

        # Implications & Biconditional
        imp_A_to_B = (not val_A) or val_B
        imp_B_to_A = (not val_B) or val_A
        bicond = (val_A == val_B)

        # Entailment check: In all models where A is True, B must also be True
        row_entails = not val_A or val_B
        if val_A and not val_B:
            all_entails = False
        if not bicond:
            all_biconditional = False

        row_str = f"| {str(A):<5} | {str(B):<5} | {str(C):<5} | {str(val_A):<6} | {str(val_B):<9} | {str(imp_A_to_B):<6} | {str(im
        # Adjust table format cleanly for A, B, C
        print(f"| {str(A):<5} | {str(B):<5} | {str(C):<5} | {str(val_A):<6} | {str(val_B):<9} | {str(imp_A_to_B):<6} | {str(imp_B_

    print("\n--- Summary Results ---")
```

```
    print("\n--- Summary Results ---")
    print(f"Does A entail B ($A \\models B$)? : {all_entails}")
    print(f"Is it Bidirectional ($A \\equiv B$ / IFF)? : {all_biconditional}")

# Example usage:
# Let A be 'A and B' and B be 'B'
print("Evaluating: A = (A and B), B = B")
check_entailment_and_biconditional("A and B", "B")
```

```
Evaluating: A = (A and B), B = B
| A | B | C | A (KB) | B (Query) | A => B | B => A | A <=> B | Entailed? |
|---|---|---|--------|-----------|--------|--------|---------|-----------|
| True  | True  | True  | True   | True      | True   | True   | True    | True      |
| True  | True  | False | True   | True      | True   | True   | True    | True      |
| True  | False | True  | False  | False     | True   | True   | True    | True      |
| True  | False | False | False  | False     | True   | True   | True    | True      |
| False | True  | True  | False  | True      | True   | False  | False   | True      |
| False | True  | False | False  | True      | True   | False  | False   | True      |
| False | False | True  | False  | False     | True   | True   | True    | True      |
| False | False | False | False  | False     | True   | True   | True    | True      |

--- Summary Results ---
Does A entail B ($A \models B$)? : True
Is it Bidirectional ($A \equiv B$ / IFF)? : False
```